

Document number: P2388R0
Date: 2021-06-15
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>
Gašper Ažman <gasper dot azman at gmail dot com>

Abort-only contract support

This paper proposes a contract support framework which is smaller than [\[P2182R1\]](#). We try to remain compatible with the plan in [\[P2182R1\]](#) to introduce contract support in phases, with the first phase (the Minimum Viable Product, MVP) being able to achieve the following goals:

1. Be a coherent whole, and provide a value to multiple groups of developers.
2. Be small enough to guarantee that it will progress fast through the standardization pipeline.
3. Be devoid of any controversial design issues; this is to obtain the maximum consensus.

Under this proposal a function containing a precondition annotation:

```
bool fun(int a, int b)
[[pre: are_compatible(a, b)]] // precondition annotation
{
    return transform(a, b);
}
```

Can render the runtime code where the precondition annotation is either ignored or produces code equivalent to:

```
bool fun(int a, int b)
{
    [&]() noexcept {
        if (are_compatible(a, b) == false) { // Note: name lookup as in function declarations
            CONTRACT_VIOLATION_HANDLER("are_compatible(a, b)");
            std::abort();
        }
    }(); // immediately invoked lambda

    return transform(a, b);
}
```

That is: a runtime check is performed, and if it returns `false` an implementation-defined message is displayed to `STDERR` and `std::abort()` is called. If any exception is thrown during the evaluation of the predicate or the logging statement, `std::terminate()` is called.

1. Overview

In this document we use the following terms recommended in [\[P2038R0\]](#) and [\[P2358R0\]](#):

Abstract machine violation

This is what the Standard defines as Undefined Behavior in Clause 4 through Clause 15, which is often called a "hard undefined behavior" or "language undefined behavior", which includes things like `++INT_MAX`, `1/0` or null pointer dereference.

BizBug

A situation where a function is invoked in a way that is disallowed by its specification; or when it returns a value or has a side effect disallowed by the specification. (This presupposes the existence of function "specification".)

Contract annotation

Declaration of a precondition or a postcondition or an assertion, such as `[[pre: i != 0]]` or `[[assert: x != y]]`. Contract annotations can express a subset of function specification.

The minimum contract support proposed in this paper consists of the following elements. The attribute-like syntax for declaring preconditions and postconditions on function declarations, and assertions inside function bodies:

```
int select(int i, int j) // first declaration
[[pre: i >= 0]]
[[pre: j >= 0]]
[[post r: r >= 0]]; // `r` names the return value
```

```

int select(int i, int j); // subsequent declarations can
                          // repeat or ignore the annotations

int select(int i, int j) // the definition
{
    [[assert: _state >= 0]];

    if (_state == 0) return i;
    else             return j;
}

```

We require that the implementation allows the translation of the program in two modes:

1. *Ignore*: compiler checks the validity of expressions in contract annotations, but the annotations have no effect on the generated binary. However, functions appearing in the predicate are ODR-used.
2. *Check_and_abort*: for each contract annotation compiler generates a runtime check. The check evaluates the corresponding predicate, and if this evaluates to `false` a *contract violation handler* is invoked.

We recommend (the ISO word "should") that the default mode is *Check_and_abort*.

What a contract violation handler does is implementation-defined subject to the following constraint. The control never leaves the violation handler. It can either stop the program or run forever (e.g., in infinite loop or hit a breakpoint). If the program exits, an implementation-defined form of the status *unsuccessful termination* is returned. We recommend (the ISO word "should") that the violation handler outputs a diagnostic message to the standard diagnostic stream and calls `std::abort()`. This corresponds to what [\[P2358R0\]](#) calls "Unspecified but never returns" and what [\[P1606R0\]](#) calls "check_never_continue".

We allow the implementations to provide a mixed mode, where some translation units are translated in *Ignore* mode and others in *Check_and_abort* mode. This may be necessary for linking the user program with compiled libraries.

This proposal does not include things like "assertion levels", "continuation mode", or the ability to install a custom violation handler.

Name lookup for preconditions and postconditions is performed as in trailing return type. Private and protected members are accessible in predicates of preconditions and postconditions of member functions.

This proposal does not allow preconditions and postconditions for lambdas. Name lookup is already problematic in lambdas in the face of lambda captures. This problem is pursued in [\[P2036R1\]](#), and until it has been solved we see no point in delaying the minimum contract support proposal.

Although it is discouraged, side effects are allowed in the predicates of contract annotations. Instead we allow the provision for the implementations to discard the side effects. This is similar to copy elision from C++03. Even in *Check_and_abort* mode, the implementation is allowed to discard or duplicate all (as opposed to 'some') side effects of the evaluated predicate, as long as this does not affect the value returned by the expression. Consider:

```

bool is_positive(int i) {
    std::printf("eval");
    return i > 0;
}

int produce() [[post r: is_positive(r)]];
void consume(int i) [[pre: is_positive(i)]];

int main() {
    consume(produce());
}

```

The above program translated in *Check_and_abort* mode is allowed to output "eval" 4, 3, 2, 1 or 0 times. Of course, the program is still aborted if the value produced by `produce()` is less than or equal to 0.

There is one case where we disallow naming the return value in the postcondition annotation: for non-definition declarations of non-templated functions with placeholder return type:

```

auto f(int i)
    [[post r: r >= 0]]; // error: illegal introduction of name `r`

auto f(int i)
    [[post r: r >= 0]] // ok: this is definition
{

```

```
    return std::abs(i);
}
```

The rationale for this decision has been provided in [\[P1323R2\]](#).

2. Motivation

The motivation for adding contract support to the language has already been provided multiple times, for example in [\[N1613\]](#), [\[N1800\]](#), [\[N4110\]](#), [\[N4075\]](#), [\[N4135\]](#), [\[P0380R0\]](#), [\[P2182R1\]](#). The motivation for starting with a minimal set of features has been provided in [\[P2182R1\]](#). In short, the goal is to minimize the risk of having the discussions and disagreements about the secondary features of contract support framework impede or prevent the addition of the primary functionality: the ability for the programmer to communicate what is considered a bug in the program. Thus, the plan for the MVP is to first determine if there exists a minimalistic subset that would be considered uncontroversial and gather consensus, polish it, and deliver reasonably fast. Only when this has been done, to start adding the secondary features on top of the stable core, and if these features fail to get consensus, the core part of contract support framework would not be in the risk of being removed or postponed.

We have observed that even the portions of [\[P2182R1\]](#) arose controversies. This paper proposes to remove these portions.

2.1. Two programming models

We have identified that there are at least two views on what a contract violation means. They both have supporters among the interested parties.

One view is that contract violation indicates a bug, and this situation is as serious as dereferencing a null pointer or making an invalid access to memory. If this situation is detected at run-time, the only acceptable default action is to immediately abort the execution of the program. This is in order to prevent any further damage.

The other view is that the contract violation is just a piece of information, usually about a bug (but not always: for instance, in unit tests a programmer may violate a contract on purpose), that can be processed in a number of ways. Program termination is one, but the program might as well just continue with the normal execution or jump to a different location, for example, by throwing an exception.

The two views conflict in whether anything other than program termination should be allowed after a runtime-confirmed contract violation. But they also have overlapping parts:

1. The common syntax for contract annotations.
2. The common understanding that contract violation, at least outside of unit tests, indicates a bug, which programmers intend to avoid or fix.
3. Even the second view allows for the mode where the program is just aborted.

We believe that this overlapping part is already useful for many groups of programmers. It allows the programmers to declare what they know to be a bug in their components. It allows tools other than compilers — such as static analyzers or IDEs — to diagnose or help diagnose bugs in programs statically. The *Check_and_abort* mode gives a guarantee to programmers that if a bug is detected at run-time the program — apart from aborting — will do no further damage. This enables UB-sanitizers to diagnose not only abstract machine violations ("hard" UB) but also contract violations: in a uniform way (provided that this way does not require the program to continue after the reported violation).

Because this proposal is — we believe — a common part of the two views, we hope that proponents of either view should find nothing unacceptable, other than that it is missing features.

3. Design rationale

3.1. Why not use attributes

An alternative syntax for contract annotations would be to use attributes:

```
// not proposed
int f(int i)
    [[pre(i >= 0)]]
    [[post(r: r >= 0)]];
```

We do not propose this for two reasons. One is social: we do not want to depart from what the previous proposal settled upon and what EWG agreed to. Second is technical. this syntax will not work for assertions in function bodies:

```
int f(int i)
{
    [[assert(i >= 0)]]; // interpreted as macro
```

```
    return i;
}
```

Here, the `assert` is interpreted as C macro and expanded, causing a compiler error. This problem is not present when identifier `assert` is followed by a colon.

3.2. Why disallow continuation (for now)

The behavior of runtime-checking, logging but not aborting has a number of surprising and non-obvious effects.

First, in case the precondition is guarding against the Abstract Machine Violation (hard UB), reaching the point of the abstract machine violation might result in what would be observed as removing the log entry for the previous contract violation. This is discussed in detail in appendix A.

Second, a mechanical transformation of every contract annotation to "check, log and continue" can introduce a new Abstract Machine Violation (hard UB) if the programmer assumes the short-circuiting behavior for subsequent contract annotations. Consider the following example.

```
int f(int * p)
[[pre: p]] [[pre: *p > 0]]
{
    if (!p)          throw Bug{}; // safety double-check
    if (*p <= 0) throw Bug{}; // safety double-check

    return *p - 1;           // (*) business logic
}
```

This program upon `p == nullptr` behaves in a way that (1) does not cause abstract machine violation and (2) guarantees that the business logic, indicated with `*`, is never reached. This happens for both *Ignore* and *Check_and_abort* mode. However, in a mode that allows the program to continue, this causes abstract machine violation upon runtime-checking the second precondition. This is explained in detail in appendix A.

We believe that "the continuation mode" is a useful feature that is necessary for some essential applications (like adding contracts in libraries in stages). Our motivation for omitting it from the MVP is the timing concerns: we want to deliver a small but relatively useful feature fast.

3.3. Why disallow throwing on contract violation (for now)

We propose to disallow throwing upon detected contract violation because it falls outside of one of the presented programming models: the one that says that it is unacceptable to allow the program with a detected bug to continue.

By disallowing throwing we also avoid exploring and describing many aspects observable behavior that this would trigger:

1. How contracts interact with `noexcept`.
2. If the precondition is evaluated before the function call or inside the function.

This makes the proposal smaller, and therefore more likely to progress faster through the WG21 process.

3.4. Why no user-provided violation handler (for now)

Not proposing the ability to install user-provided violation handlers is again motivated by timing constraints. This way we avoid the necessity to specify the interface for the violation handler and its constraints.

However, the way we specify the handler (mostly implementation defined) allows for things like programmer installing a callback in an implementation-defined way.

3.5. Why allow side effects in predicates

We intuitively expect that predicates in contract annotations are referentially transparent; that is, they have no side effects and they always return the same value for the same values of input parameters. This expectation is reflected in terms like "if the precondition *holds*". However, it is impossible to enforce such constraint in an imperative language like C++.

It is often impossible for programmers to even know if the function they use has any side effects. For instance, the specification of `std::vector<T>::size()` `const` does not prevent the implementations from performing side effects, such as logging.

Some side effects are practical to have, and they do not affect the reasoning about predicates in the mathematical sense. These side effects include:

1. Logging, which never affects subsequent computations.
2. Modifying private mutable data members for the purpose of caching function results.
3. Using mathematical functions from `<cmath>`, which store error results in global (thread-local) variable `errno`.
4. Performing scoped locking inside the function, which may affect the execution of other threads.
5. Causing a contract violation handler when runtime-checking the precondition of the function called in the predicate.

Our choice follows the existing practice with `assert()`: it allows side effects, but a lot of advice comes with it, saying that side effects in the predicate cannot be relied upon.

[\[P0542R5\]](#) specified that invoking any side effect inside a contract annotation predicate is undefined behavior. This allowed certain practical optimizations, such as not performing two consecutive evaluations of the same predicate, which is often the case when postcondition of one function is the same as the precondition of another function. It also allowed checking the same predicate twice, for instance once inside the function body, and the second time in the calling context.

While we drop this undefined behavior, we allow similar effects, by explicitly listing what transformations the compiler is allowed to perform. Namely, it can remove or duplicate all (not some, all of them) side effects associated with evaluating the predicate from a contract annotation. From this it follows that if you know what value would be returned by the predicate without evaluating the predicate, you can just use this value rather than evaluating the predicate. This means that when the following predicate is used in a contract annotation:

```
bool is_positive(int i) {
    std::printf("eval");
    return i > 0;
}
```

It can be replaced with just `i > 0`.

3.6. Why include side effect elision in the MVP

First, it cannot be added later, because then it would be a breaking change. User may start to rely on the mandated side effects in *Check_and_abort* mode, as per the Hyrum Law, and these would suddenly disappear.

Second, it gives a strong encouragement to the users not to put side effects in their predicates. Their side effects may disappear, even in *Check_and_abort* mode.

3.7. Why allow access to private and protected members

Programming guidelines often recommend that in contract predicates of public member functions one should only use the public interface of the class. This is in case when the class user needs to manually check if the contract is satisfied for an object whose state is not known. However, this is only a guideline, and enforcing it statically would break other use cases that do not subscribe to the above advice. This has been described in detail in [\[P1289R1\]](#), and in fact adopted by EWG.

3.8. Why not mandate a default translation mode

We recommend that the default mode is *Check_and_abort*, but we do not require this of the implementations. The reason for that is that we believe that it is not possible to mandate this behavior in this International Standard. This has been discussed in detail in [\[P1769R0\]](#).

4. Future compatibility

While we propose to drop a number of features from the MVP, this proposal does not close the door for adding them in the future, once (if) the MVP has been agreed upon and baked.

The ability to install a custom violation handler (along with the handler's interface) can be provided as a future extension, with the behavior mandated by the MVP being the semantics of the default violation handler.

The ability to continue after logging the contract violation has two use cases:

1. Apply the "continue" semantics for all contract annotations when running the variant of a program with contract checking enabled for the first time.
2. Applying the "continue" semantics selectively only for the newly added contract annotations.

In the first case, the application of the "continue" semantics is not a default behavior, and would require that a person who assembles the program instructs the compiler to use the special behavior. This can be added in the future as a third mode of translating the source code with contract annotations (in addition to "ignore" and "check_and_abort").

In the second case, there is a need to discriminate the newly added contract annotations from the stable ones. This would require some additional syntax to mark such annotations, for instance:

```
int f(int * p)
  [[pre: p]]           // stable annotation
  [[pre: *p > 0; new]] // new annotation
;
```

This can be added in the future by introducing a new syntax for the newly added annotations and allowing the programmer to control separately what runtime code should be generated from these "new" annotations.

The syntax space for additional information in contract annotations is quite broad. The alternatives include:

```
[[post r: r > 0: new]]
[[post{new} r: r > 0]]
```

Regarding the throwing violation handlers, the only known and well explored use case is for "negative" unit-testing. We note that this ability would only be used in special programs: ones that execute unit tests. It would also require of the tested functions not to be marked as `noexcept`. For this special case it seems reasonable to expect of the person that assembles the program that they instruct the compiler in an explicit way that a unit-test program is built. This ability could be added as a future extension by introducing a yet another translation mode where exceptions thrown from violation handlers are not immediately turned into a call to `std::terminate()` (but they can still be turned into `std::terminate()` when `noexcept` functions are executed in unit-test programs).

This might look like a lot of modes of translation, but it should be kept in mind that the perspective of a person that looks at the code will not have modes: a declaration starting with `[[pre:` is always a precondition: something that evaluates to `true` in correct programs. The modes will be visible only to the person assembling the program, and in this case, having a lot of fine grained control is desired.

The point of this section is to illustrate that dropping features like violation handlers, continuation after a failed runtime check or throwing violation handlers from the MVP does not build any technical barriers that would prevent the addition of these features in the future revisions of the contract support framework. We also expect that once the syntax for declaring contract annotations has been standardized, compiler vendors will offer non-standard extensions that will allow the users to experiment with additional features and become a basis for the future standardization.

5. Open Issues

5.1. Const qualification

Ideally, we would like contract predicates to be referentially transparent (have no side effects, depend on nothing else but the objects directly referenced in the expression). This goal is not realistically attainable. We could get closer, however, if we required that all objects referenced in the predicate are treated as if they were `const`. Thus, only operators and functions (including member functions) that take arguments by value or a reference to `const` are allowed.

This could cause some discrepancies in case we have two overloads: one for type `T const&` and the other for `T&` doing different things:

```
struct X
{
  bool is_fine() const { return /* impl 1 */; }
  bool is_fine()      { return /* impl 2 */; }

  friend void fun(X& x)
    [[pre: x.is_fine()]] // uses impl 1
  {
    [[asset: x.is_fine()]]; // uses impl 2
  }
};
```

Also, there exist referentially transparent functions that do not mark their reference arguments as `const`. While programmers are often advised to mark any functions that do not modify their arguments by contract as `const`, this is just a device, and does not have to be followed. And it is not clear if such functions should be excluded from contract predicates.

5.2. Interaction with `constexpr` functions

Ideally, we would like the constant evaluation of a `constexpr` function when any contract annotation is violated to make the program ill-formed. However, we are not sure if this is implementable. It is still to be decided if the violated contracts should turn into an ill-formed program when the translation mode is *Ignore*.

6. Wording

TBD

7. Acknowledgments

Daveed Vandevorde offered useful feedback on the syntax of contract annotations.

8. References

- [N1613] — Thorsten Ottosen, "Proposal to add Design by Contract to C++" (<http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2004/n1613.pdf>).
- [N1800] — Lawrence Crowl, Thorsten Ottosen, "Contract Programming For C++0x" (<http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2005/n1800.pdf>).
- [N4110] — J. Daniel Garcia, "Exploring the design space of contract specifications for C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4110.pdf>).
- [N4075] — John Lakos, Alexei Zakharov, Alexander Beels, "Centralized Defensive-Programming Support for Narrow Contracts(Revision 6)" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4075.pdf>).
- [N4135] — John Lakos, Alexei Zakharov, Alexander Beels, Nathan Myers, "Language Support for Runtime Contract Validation (Revision 8)" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4135.pdf>).
- [P0380R0] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "A Contract Design" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0380r0.pdf>).
- [P1494R1] — S. Davis Herring, "Partial program correctness" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1494r1.html>).
- [P2339R0] — Andrzej Krzemiński, "Contract violation handlers" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2339r0.html>).
- [P2358R0] — Gašper Ažman, John McFarlane, Bronek Kozicki, "Defining Contracts" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2358r0.pdf>).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P1289R1] — J. Daniel Garcia, Ville Voutilainen, "Access control in contract conditions" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1289r1.pdf>).
- [P1323R2] — Hubert S.K. Tong, "Contract postconditions and return type deduction" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1323r2.html>).
- [P1606R0] — Joshua Berne, "Requirements for Contract Roles" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1606r0.pdf>).
- [P1769R0] — Ville Voutilainen, Richard Smith, "The «default» contract build-level and continuation-mode should be implementation-defined" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1769r0.html>).
- [P2038R0] — Andrzej Krzemiński, Ryan McDougal, "Proposed nomenclature for contract-related proposals" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2038r0.html>).
- [P2182R1] — Andrzej Krzemiński, Joshua Berne, Ryan McDougall, "Contract Support: Defining the Minimum Viable Feature Set" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2182r1.html>).
- [P2036R1] — Barry Revzin, "Change scope of lambda *trailing-return-type*" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2036r1.html>).

Appendix A. Open issues with continuing violation handlers

We are aware of two unintuitive consequences of continuing violation handlers. First, in case the precondition is guarding against the abstract machine violation (hard UB), logging and then reaching the point of the abstract machine violation might result in what would be observed as removing the log entry for the contract violation. This has been described in detail in [P2339R0] as well as in [P1494R1]. This is no worse than disabling runtime checking altogether (which is uncontroversial), but can really fool whoever troubleshoots the bug: we see no contract-violation log entry, so we think control never reached this point, even though it did. Thus, the continuing violation handler has the potential to deceive whoever uses contract violation logs. There is no agreement on how realistic the possibility of encountering this effect is. The solution presented in [P1494R1] has the potential to address the above issue. But until this is explored, any wider contract proposal that allows continuation would be blocked on [P1494R1].

Second, the continuation mode can introduce new abstract machine violations. Going back to the example provided earlier:

```
int f(int * p)
  [[pre: p]]
```

```

    [[pre: *p > 0]]
{
    if (!p)                // safety double-check
        throw Bug{};

    if (*p <= 0)            // safety double-check
        throw Bug{};

    return *p - 1;
}

```

If this function is invoked in a program that doesn't runtime-check contract annotations, it behaves in a tolerable way for `p == nullptr`: it throws an exception. But when runtime checking is enabled and the program is allowed to continue after the failed check, this code is equivalent to:

```

int f(int * p)
{
    if (!p)                // (1)
        log_violation();

    if (*p <= 0)            // (2)
        log_violation();

    if (!p)                // safety double-check
        throw Bug{};

    if (*p <= 0)            // safety double-check
        throw Bug{};

    return *p - 1;
}

```

If `p` happens to be null, check (1) will fail and the violation will get logged. The program will proceed to check (2) and there, it will dereference the null pointer causing an Abstract Machine Violation (hard UB). The key observation here is that the defensive checks that throw exceptions (or return) have the "short circuiting" property: if one fails, the subsequent ones are not executed:

```

int f(int * p)
{
    if (!p)
        throw Bug{};

    if (*p <= 0)            // null `p` never dereferenced
        throw Bug{};

    return *p - 1;
}

```

Short-circuiting is also guaranteed for subsequent precondition annotations, provided that the contract violation logging ends in calling `std::abort()`. But short-circuiting is gone, when the handler allows the program flow to continue.

Splitting a precondition into smaller chunks is used for at least two purposes:

1. Obtaining as fine-grained information as possible from contract violation logs.
2. Differentiating cheap and expensive checks, for the purpose of controlling their behavior separately.

Until this problem is addressed, the continuation after a runtime-evaluated check is a potentially dangerous feature that can introduce an Abstract Machine Violation (hard UB) on top of a BizBug (a programmer bug). While this problem may be solvable, the analysis and solution will take time, which will delay the adoption of the minimum contract support if it allows the continuation.